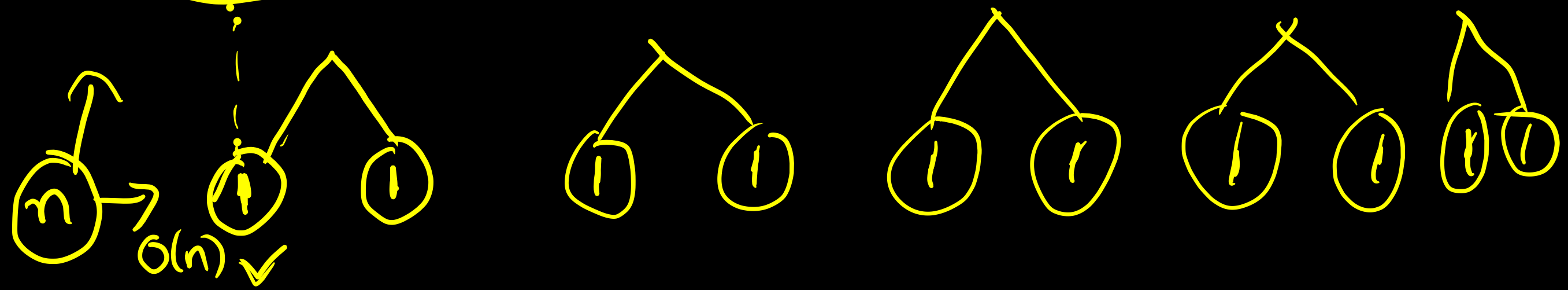
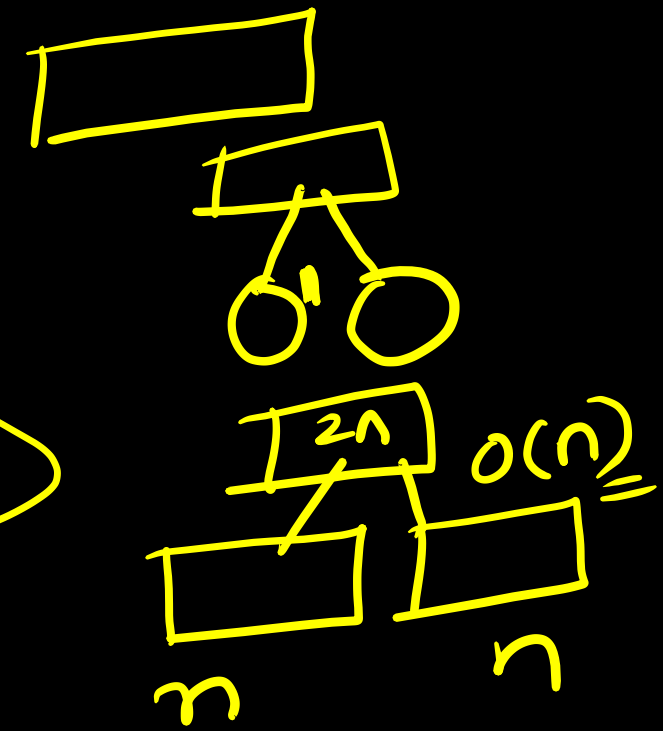
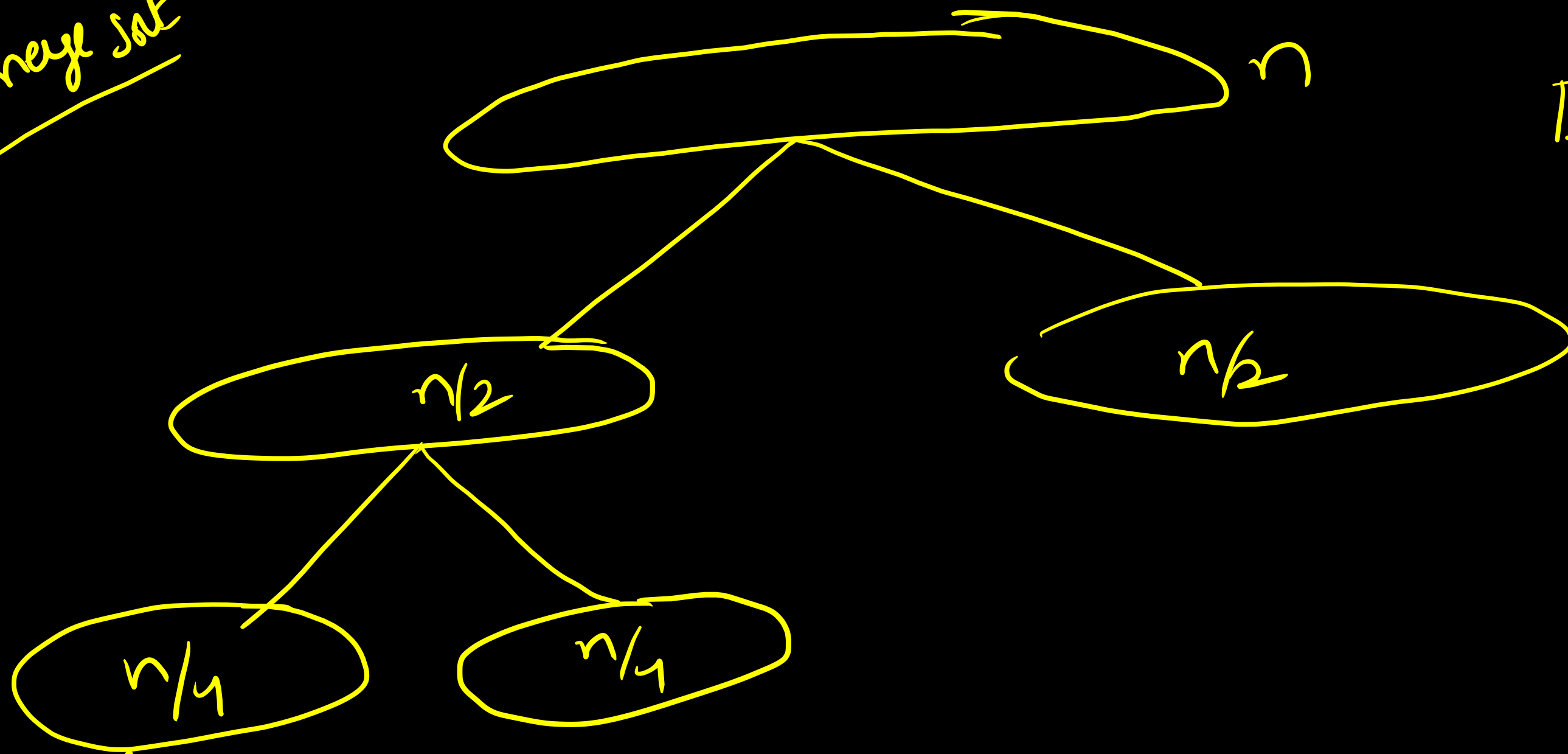
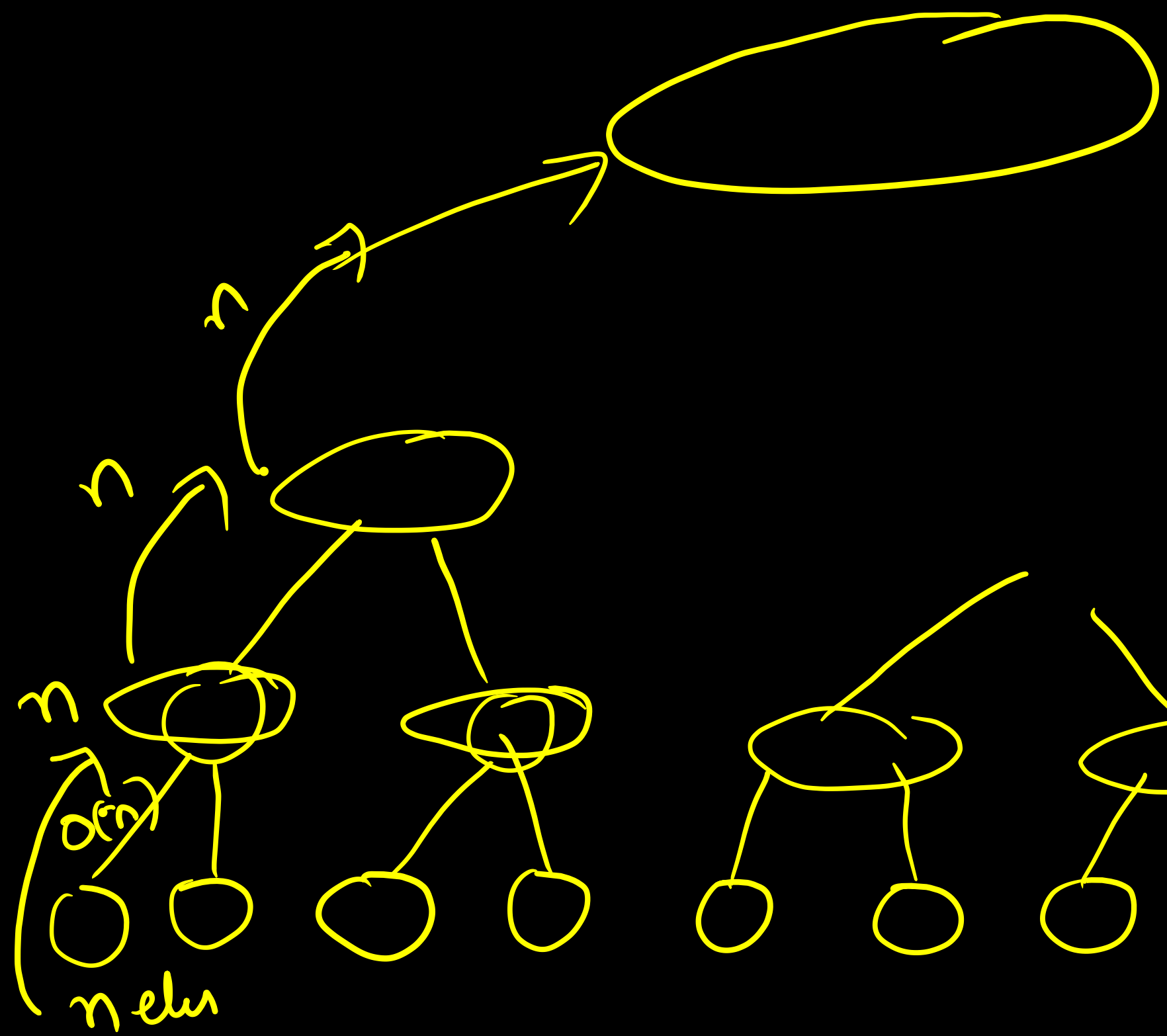
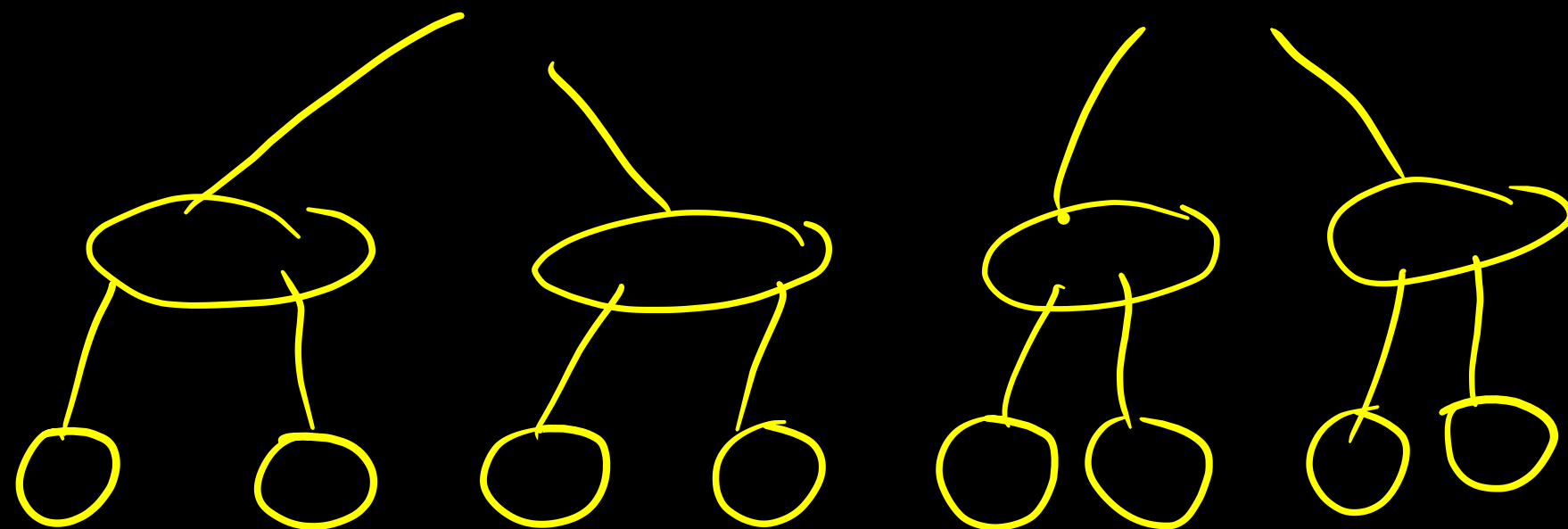


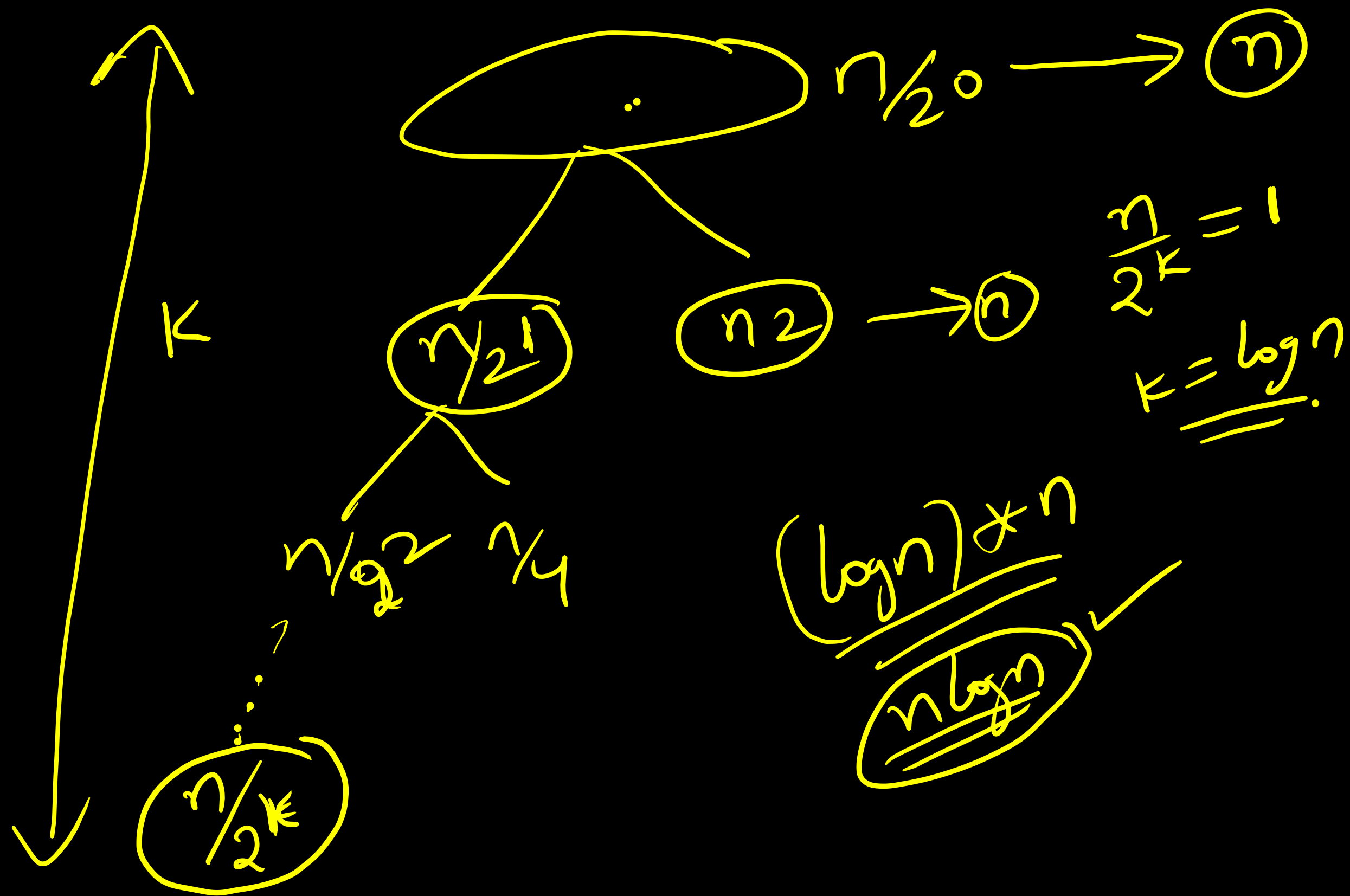
merge sort





at every level $O(n)$
how many levels





merge sort



Sorting

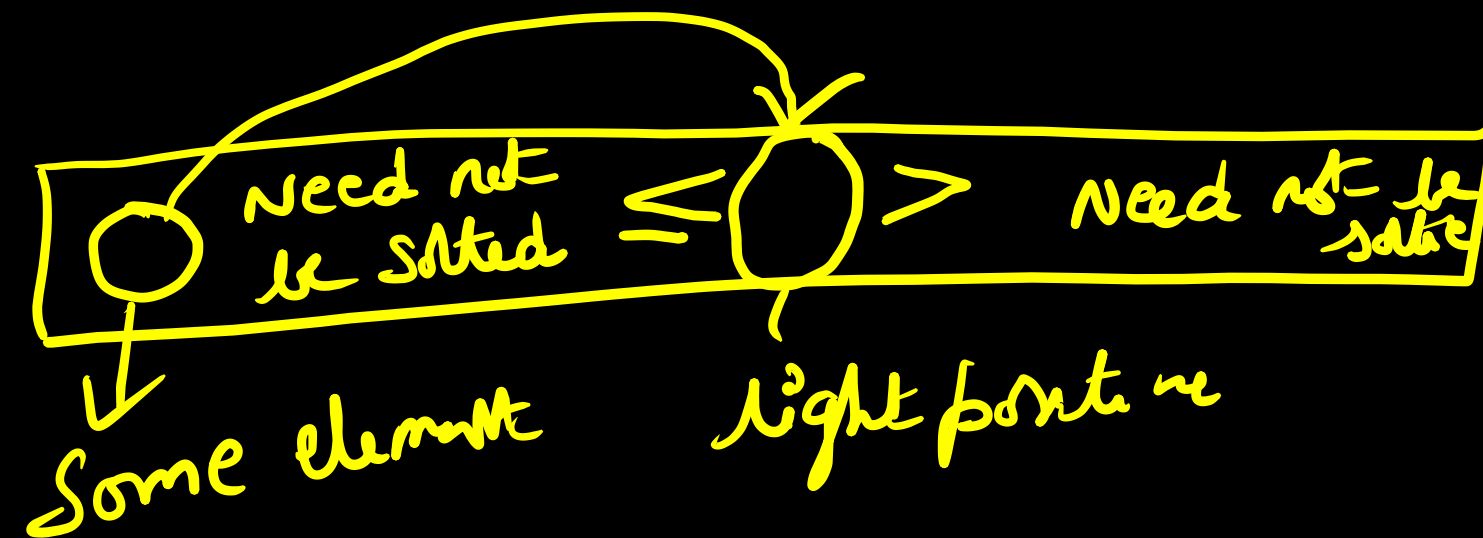
in new node → Single element array is already sorted.

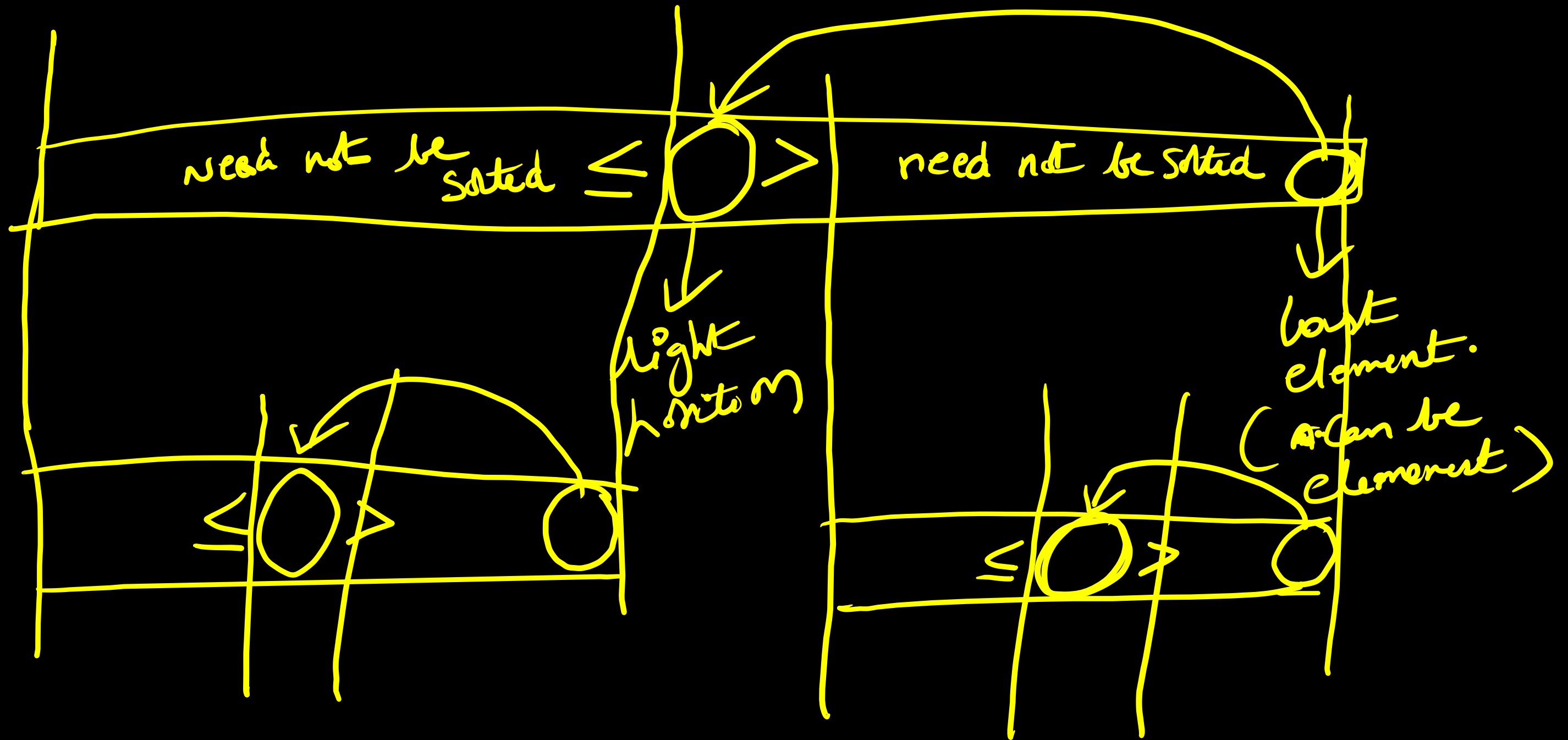


use recursion
merging to
achieve the sorting.



Quick sort $\rightarrow$ uses divide and conquer.



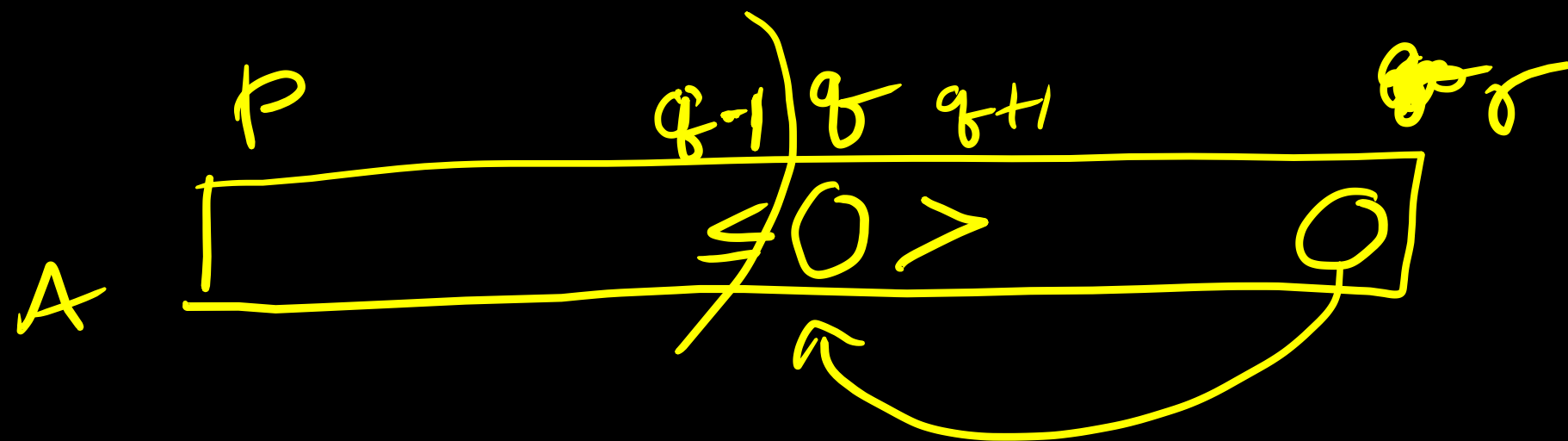


Divide: Partition (rearrange) the array $A[p \dots r]$ into two (possibly empty) subarrays $A[p \dots q-1]$ and $A[q+1 \dots r]$

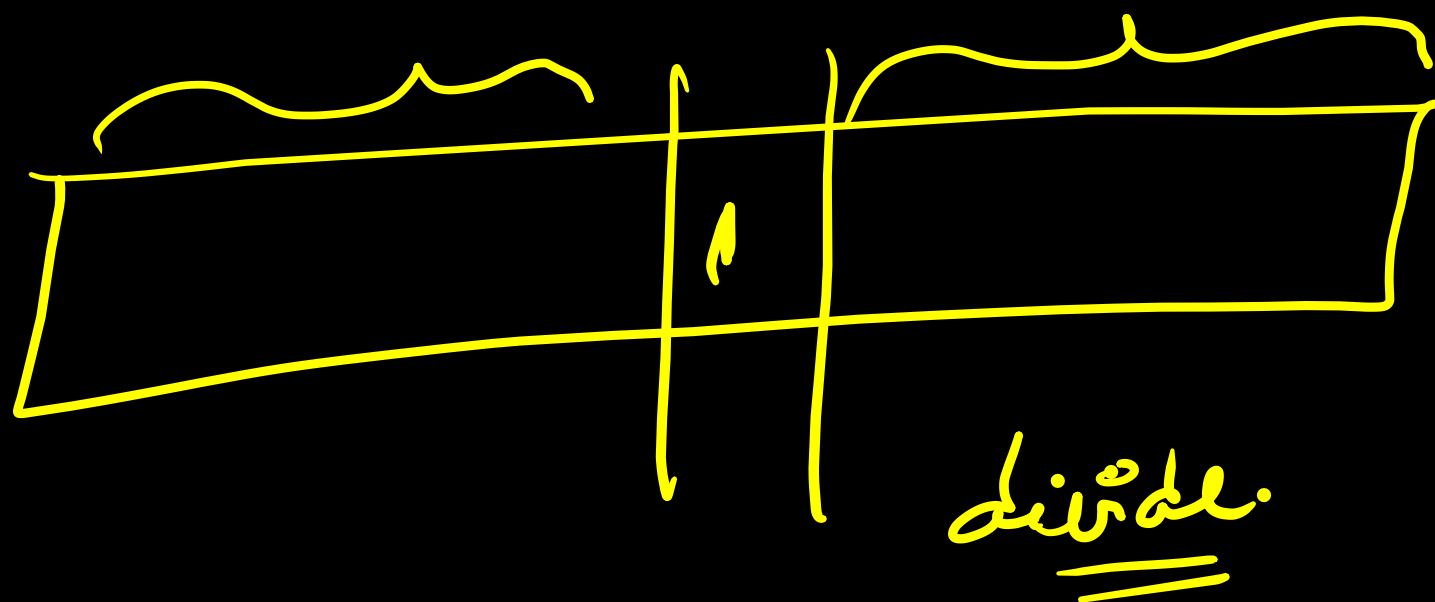
such that each element of $A[p \dots q-1]$ is less than or equal to $A[q]$, which is, in turn, less than

or ~~equal to~~ ~~equal to~~ each element of $A[q+1 \dots r]$.

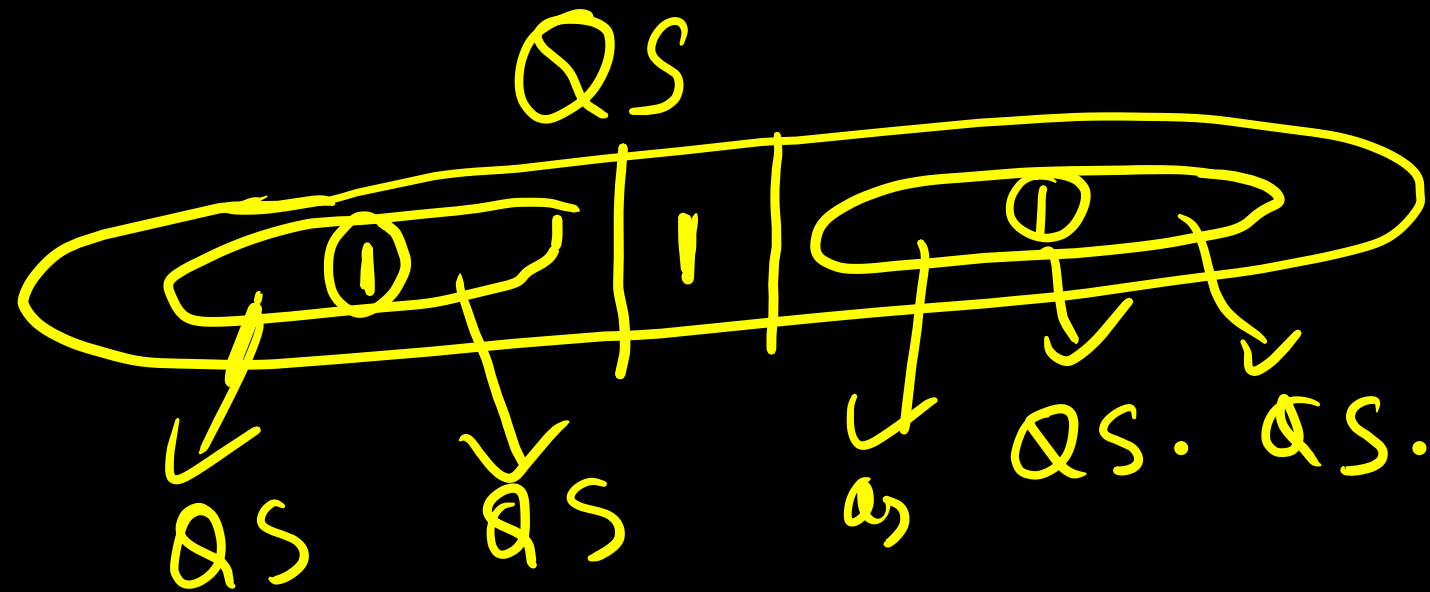
Compute the index q as a part of the partitioning algo.



Divide



Conquer: solve the two subarrays $A[p \dots q-1]$ and $A[q+1 \dots r]$ by recursively calling QS.



Combine: Since the subarrays are sorted
in place, no need to combine them

D - Divide

A

C - Conquer

~~C - Combine~~

C - Combine (Something)

if S is in place

MS
was
out of
place

MS

{

→ divide

MS

MS

→ Conquer

merge.

→ Combine

extra
space

y

MS

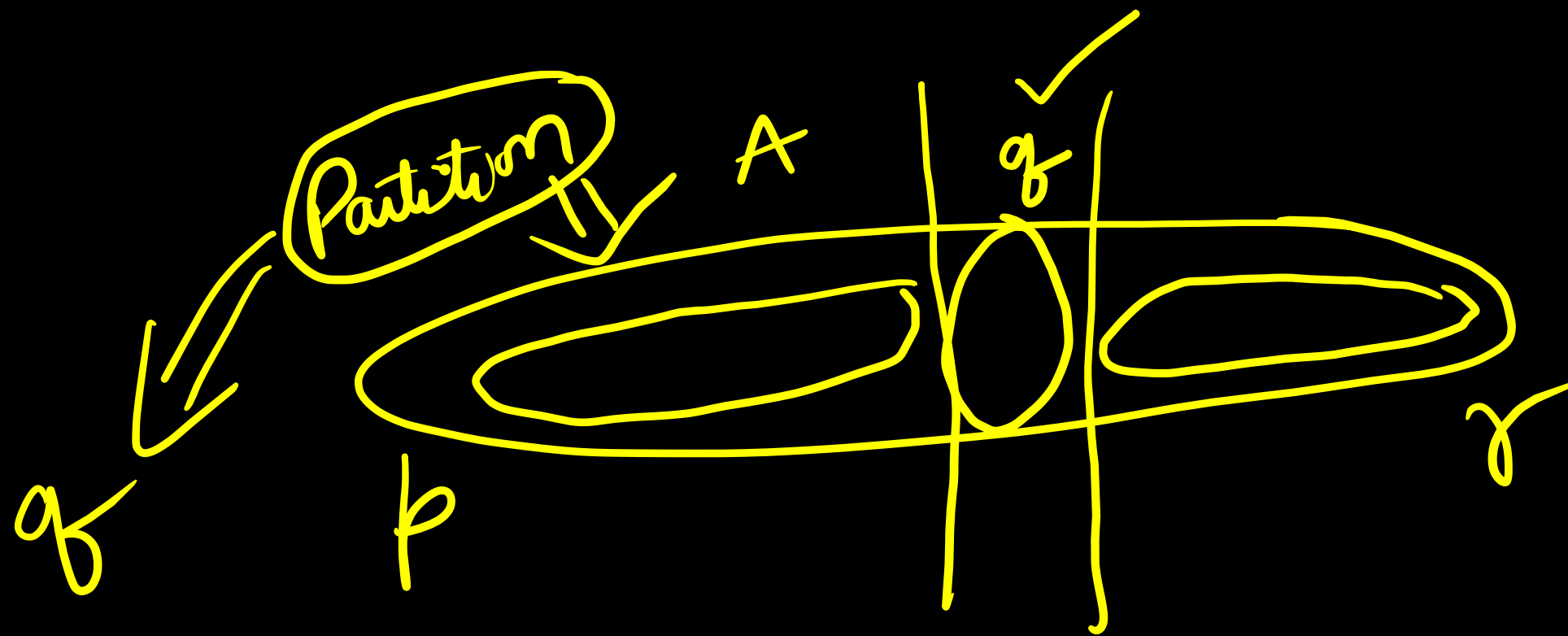
A

OCN

L

RT

AT



partition is the
heart of quick sort.

$$A[p \text{ to } q-1] \leq A[q]$$

$$A[q+1 \text{ to } r] > A[q]$$

Quick sort (A, p, r)

{ if ($p < r$)

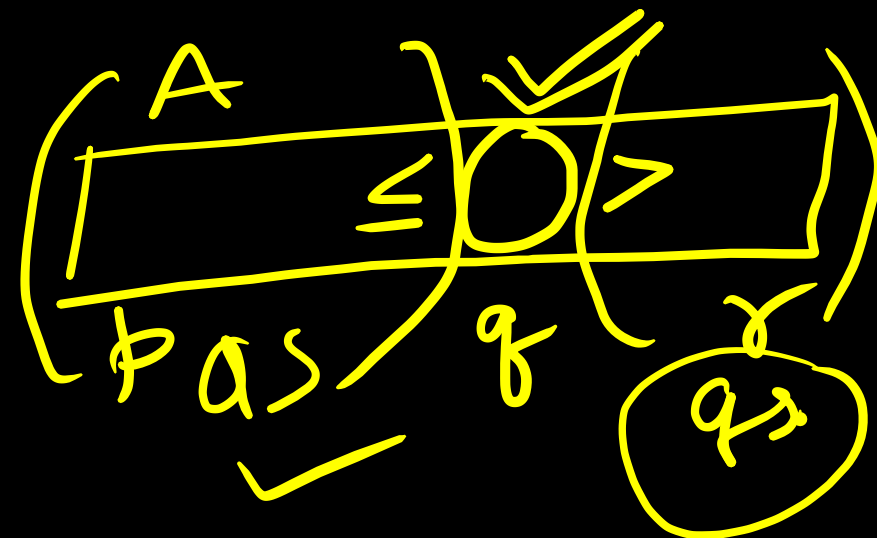
→ two elements ✓

{ $q = \text{partition}(A, p, r)$

Quick sort ($A, p, q-1$)

Quick sort ($A, q+1, r$)

Partition
part
}



$q \Rightarrow$ inserted and so not included in recursive calls again.

Partition is putting one element in its place
one at a time.

Partition(A, p, r)

$\{ \rightarrow x = \underline{A[r]}$

$\rightarrow i = p - 1$

$\rightarrow \text{for } (j = p \text{ to } r - 1):$

$\{ \rightarrow \text{if } (A[j] \leq x)$

$\{ \rightarrow \underline{i = i + 1}$
exchange $A[i]$ with $A[j]$

$\}$

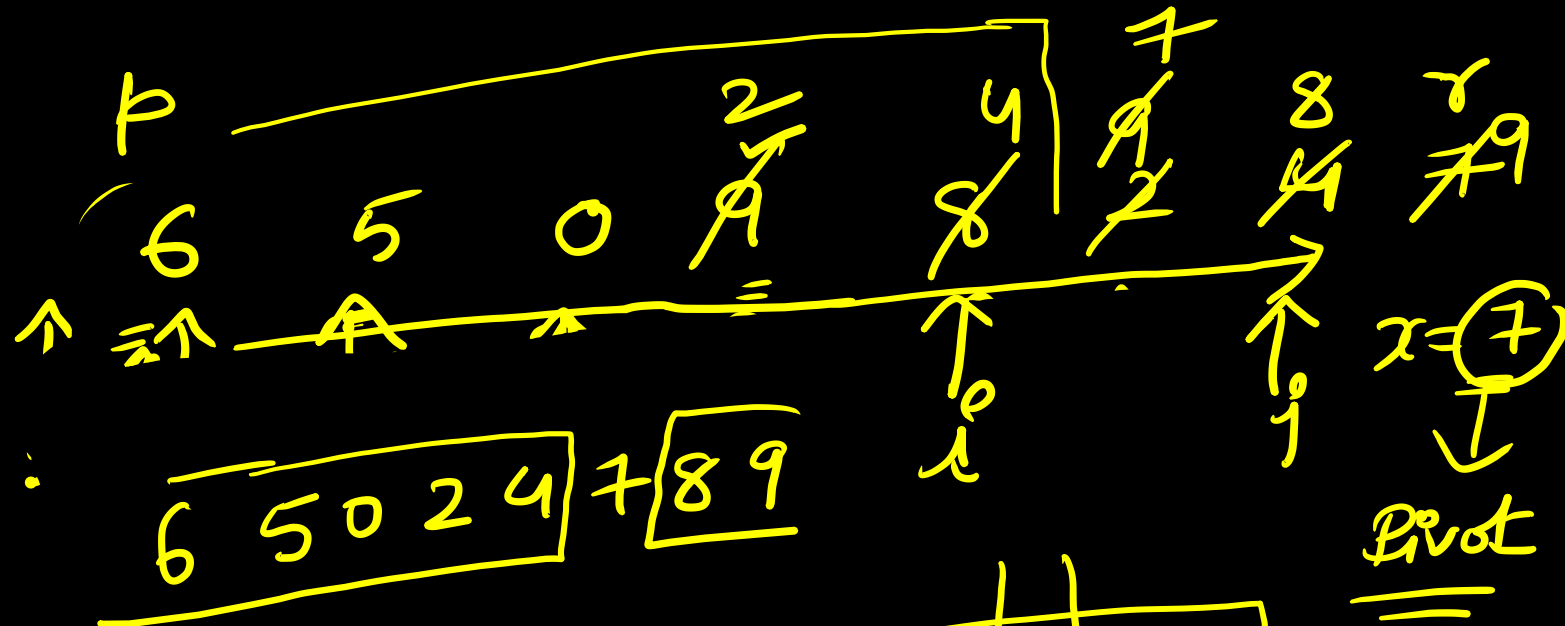
$\rightarrow \text{exchange } (A[i + 1] \text{ with } A[r])$

$\rightarrow \text{return } i + 1$

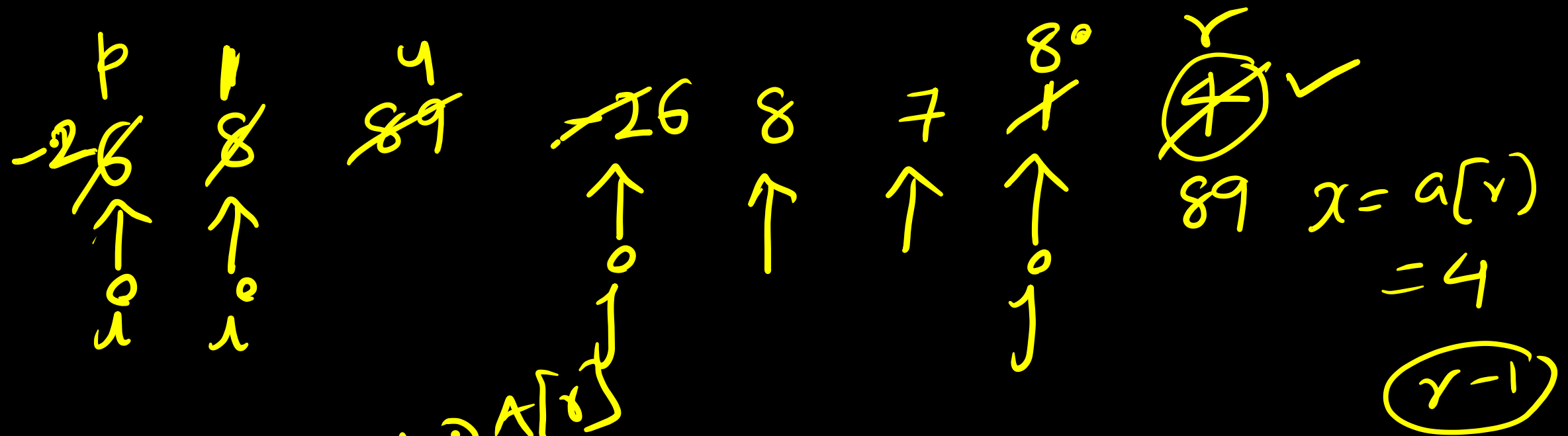
p

9 6 5 0 8 2 4 7

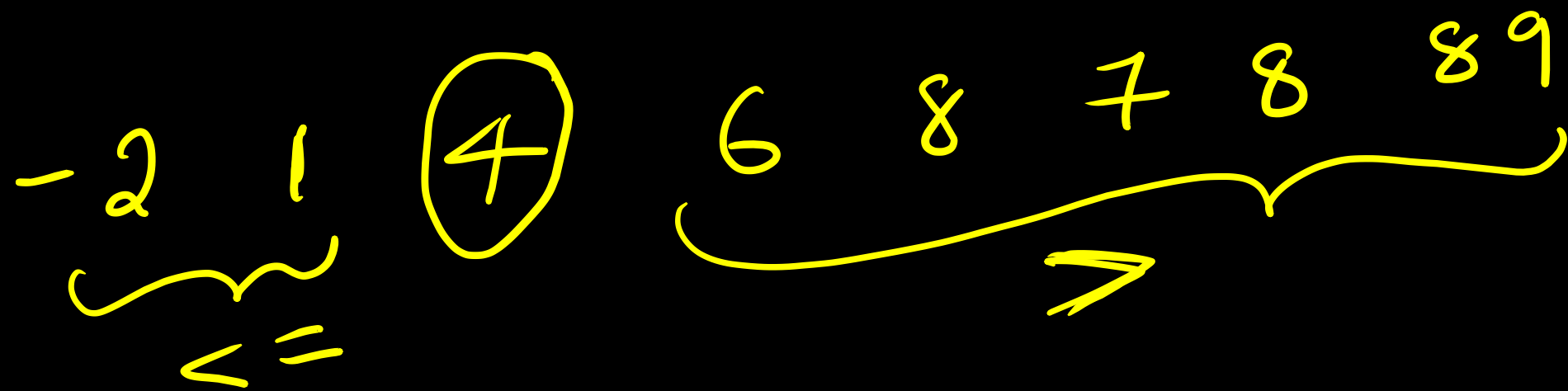
x = 7

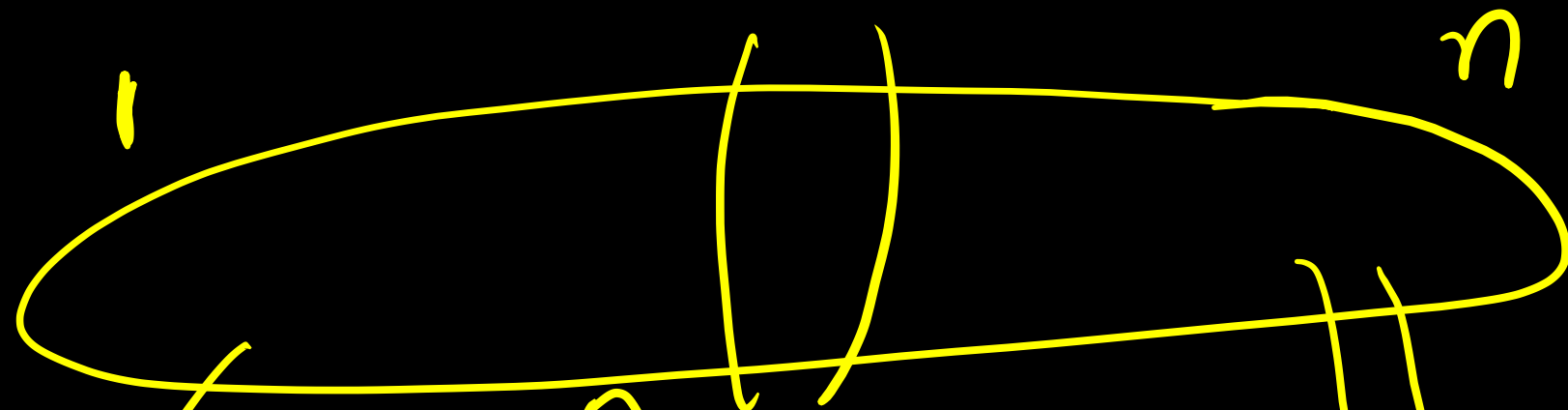


if j finds any element ≤ 7 , it moves to first hole



$A[i+1] \rightarrow A[i]$
 $i++$
 $A[i] \quad A[j]$

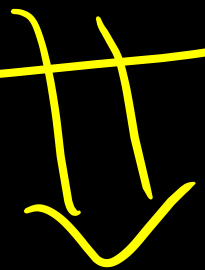




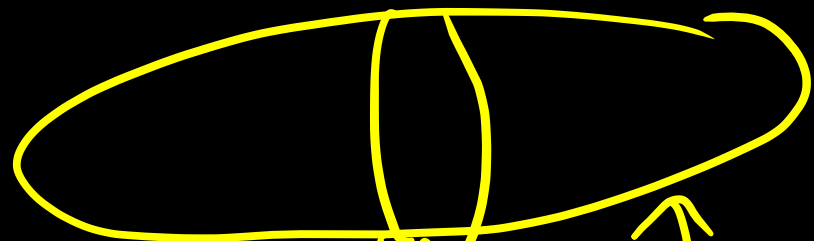
QS



partition = $\left(\frac{n}{2}\right)$



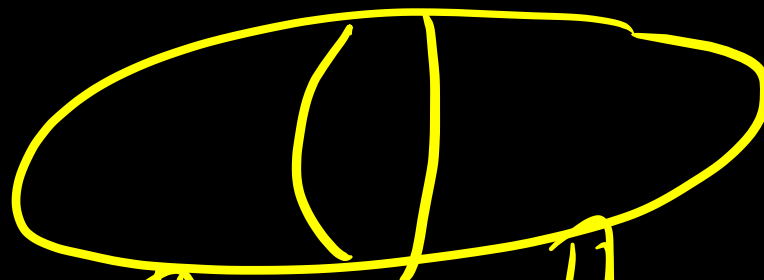
QS.



QS

part.

QS

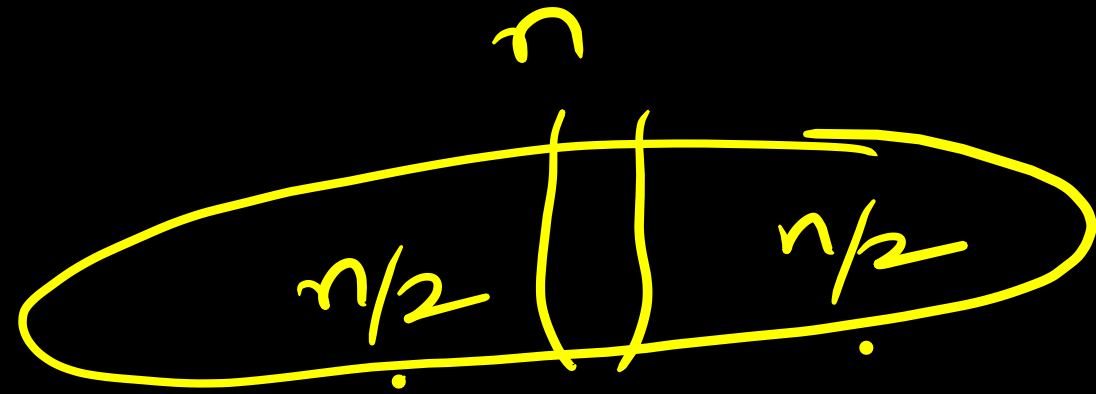


QS

QS.
=

Analysis of QS:-

BC =



TC = for partition.

QS(n)

{

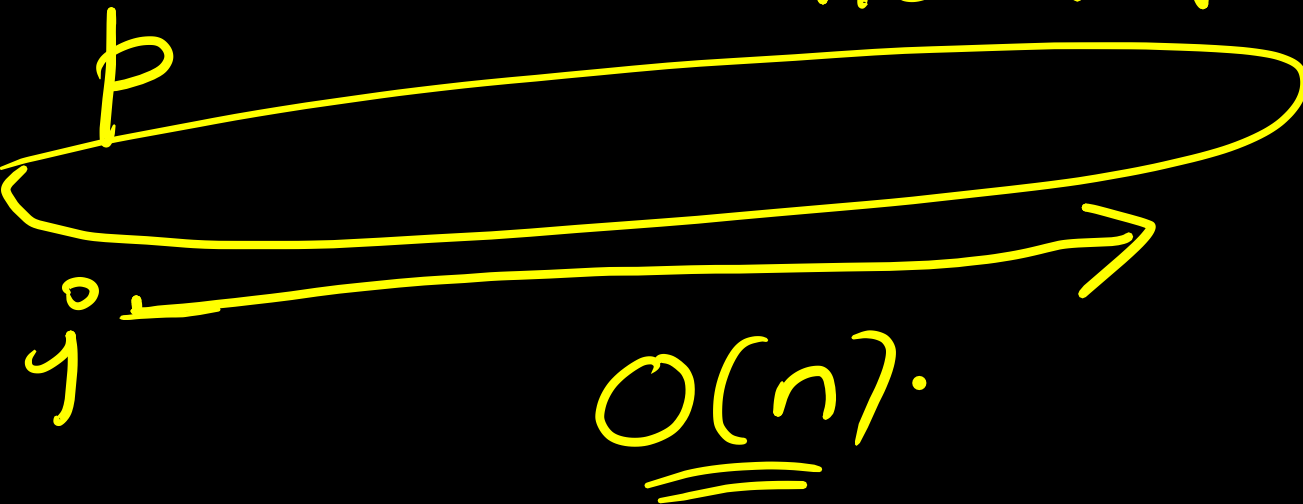
partition $\rightarrow O(n)$

QS - $n/2$

QS - $n/2$

QS

}



Berk's Case

$$QS(n) \Rightarrow T(n)$$

{

partition — $O(n)$;

$$QS(n/2) \Rightarrow T(n/2)$$

$$QS(n/2) \Rightarrow T(n/2)$$

}

$$\begin{aligned} T(n) &= 2T(n/2) + c\sqrt{n} \\ &= \underline{\underline{O(n \log n)}}. \end{aligned}$$

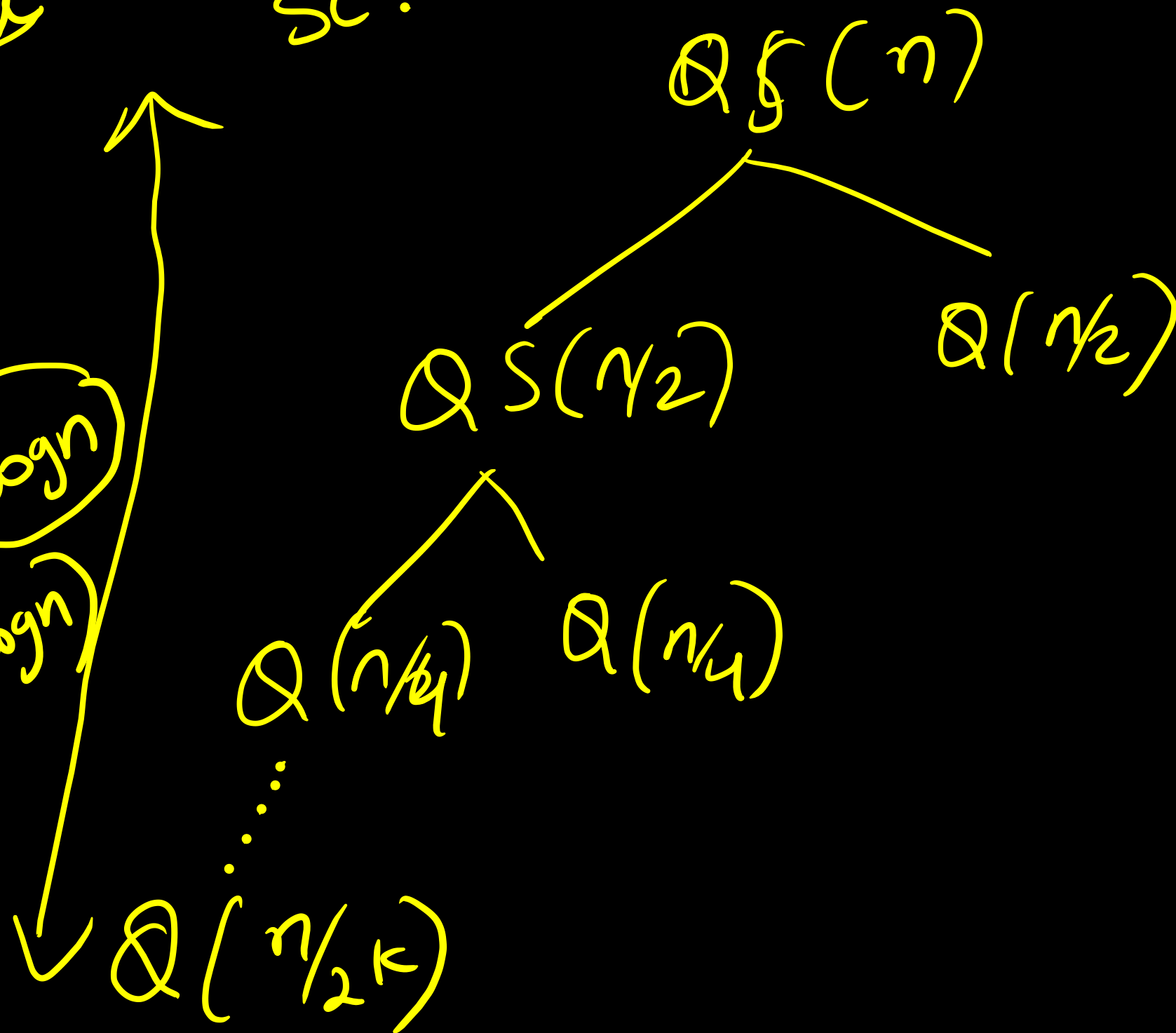
Best Cor:

SC:

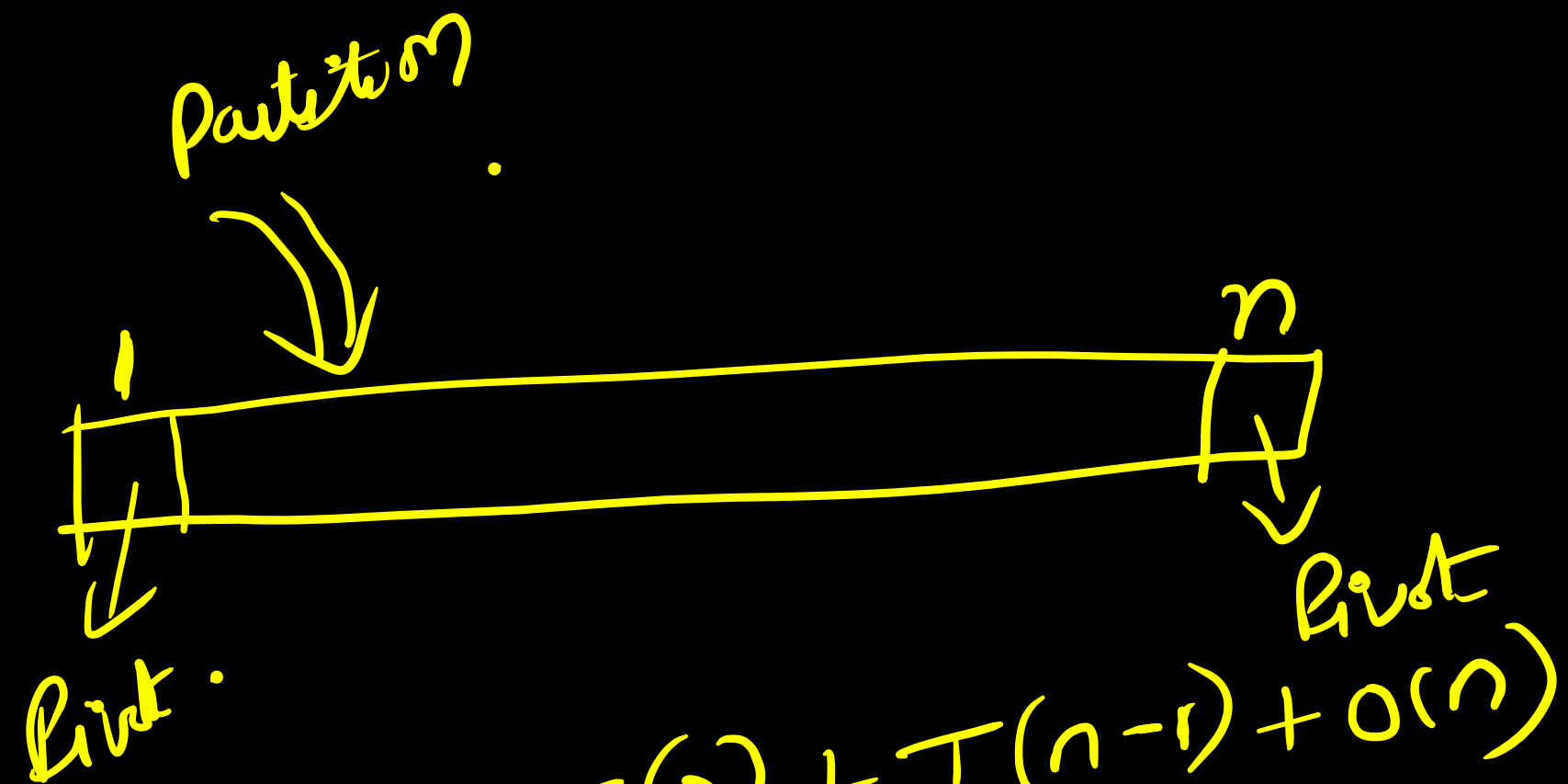
BC TC: $O(n \log n)$
SC: $O(\log n)$.

$$\begin{aligned} n/2^k &= 1 \\ n &= 2^k \\ k &= \log n. \end{aligned}$$

SC = $O(\log n)$



Worst Case

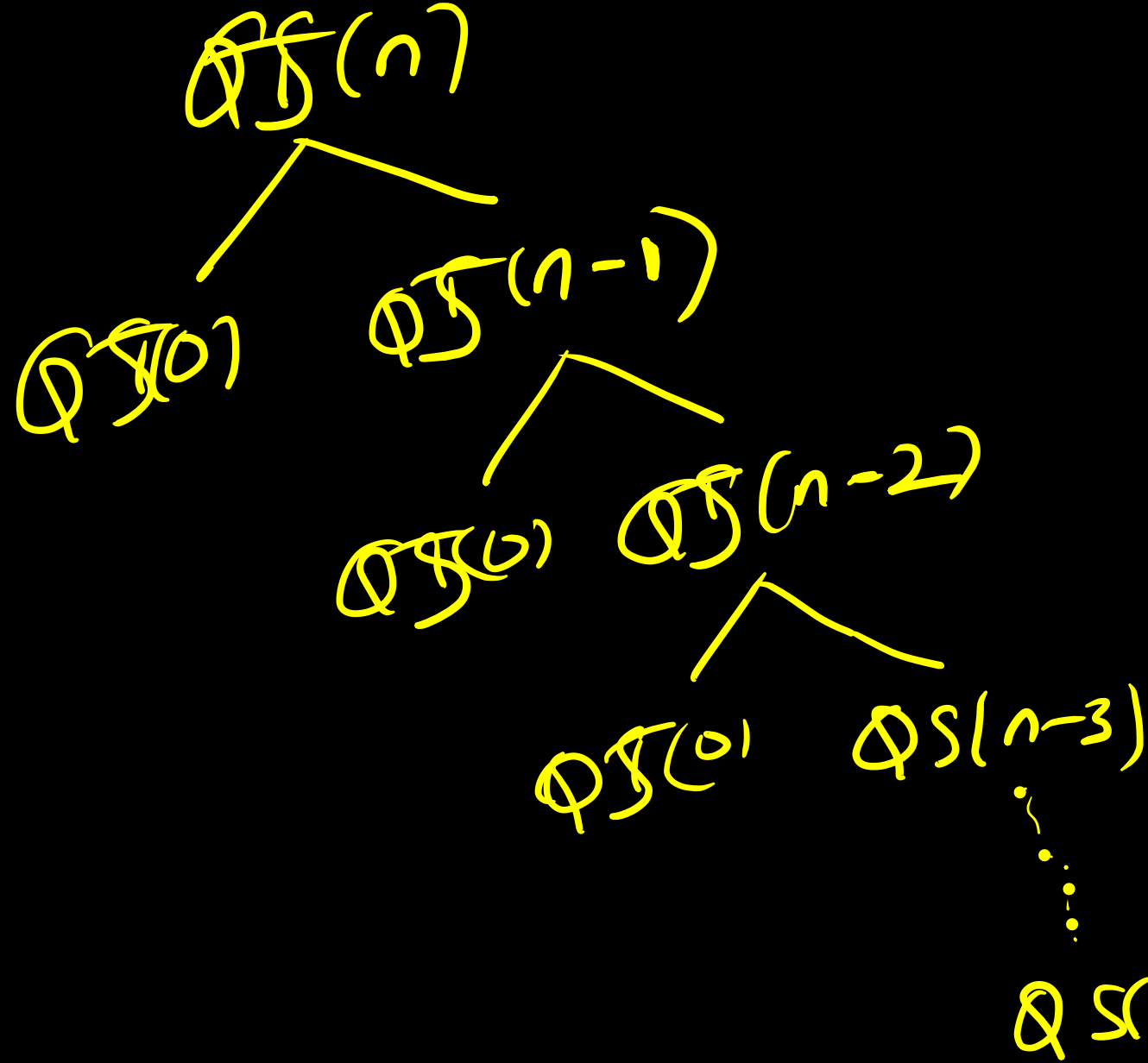


$$\begin{aligned} T(n) &= T(0) + T(n-1) + O(n) \\ &= T(n-1) + \underline{cn}. \end{aligned}$$

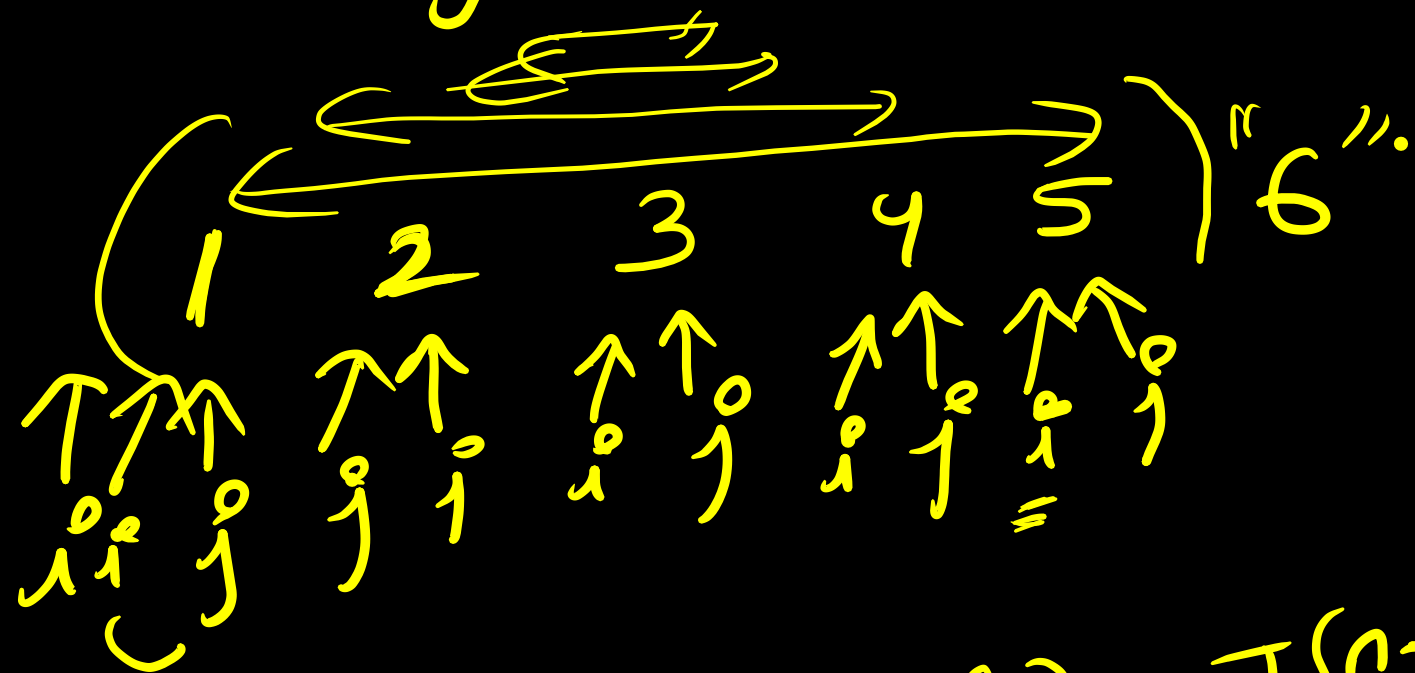
$\Downarrow$
Substitution
 $= O(n^2)$

Work Case : SC

WC SC = $O(n)$



what happens if QS is applied on an array
sorted in ascending order



$x = 6$

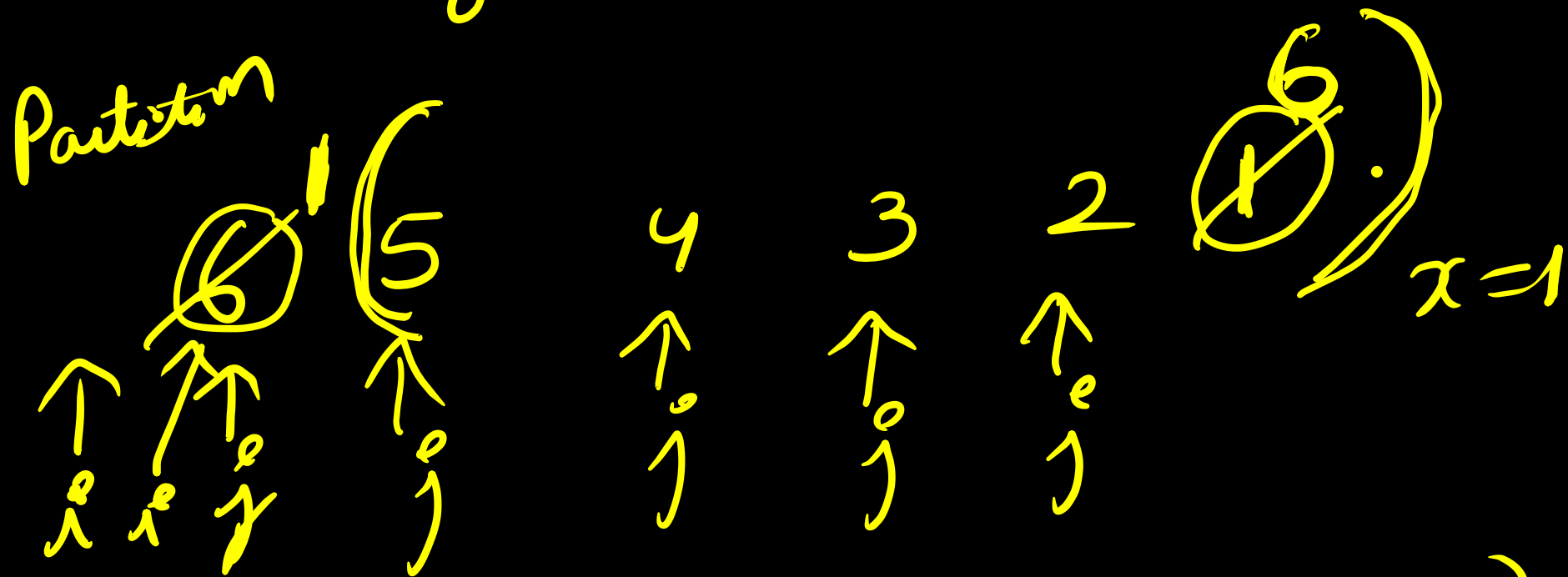
partition.

$$T(n) = T(n-1) + T(0) + O(n)$$

$$T(n) = O(n^2)$$

$$S = O(n)$$

descending order array:

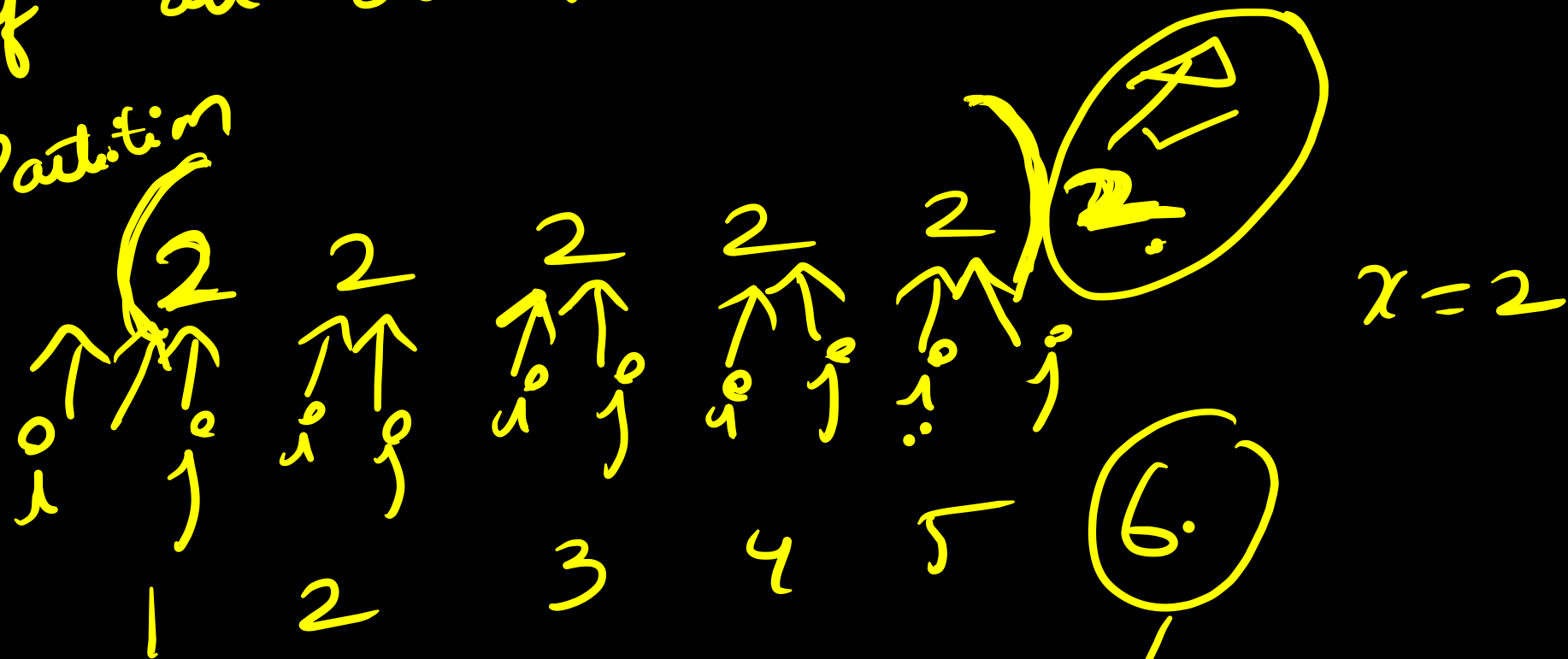


$$T(n) = T(0) + T(n-1) + O(n)$$
$$= O(n^2)$$

$$SC = O(n).$$

if all elements are same

partition



(1 5)

$$\begin{aligned} T(n) &= T(0) + T(n-1) + O(n) \\ &= O(n^2) \end{aligned}$$

Let us say each split is 1:9

~~$T(n)$~~ $T(n) \rightarrow O(n)$ —

$T(n/10) \quad T(9n/10) \rightarrow O(n)$

$T(n/100) \quad T(9n/100) \quad T(81n/100) \rightarrow O(n)$

$T\left(\frac{n}{\left(\frac{10}{9}\right)^k}\right)$

$$\frac{n}{\left(\frac{10}{9}\right)^k} = 1$$

$$\Rightarrow k = \log_{10/9} n$$

$$= O(\log n)$$

$$O(n \log n)$$

1:9

↙

1:99

↙

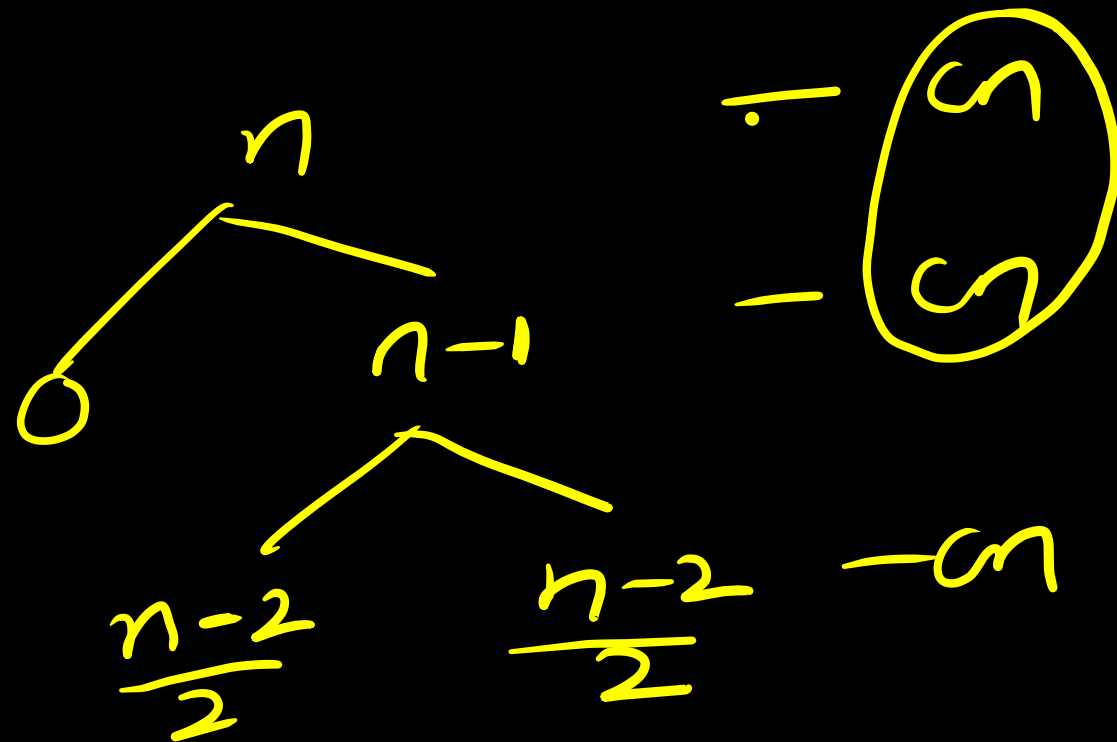
1:999

↙

1:9999

$O(n \log n)$

if alternatively worst and best happened



$$T(n) = cn + cn + (n + 2T(\frac{n-2}{2}))$$

$$\leq 3cn + 2T(\frac{n}{2})$$

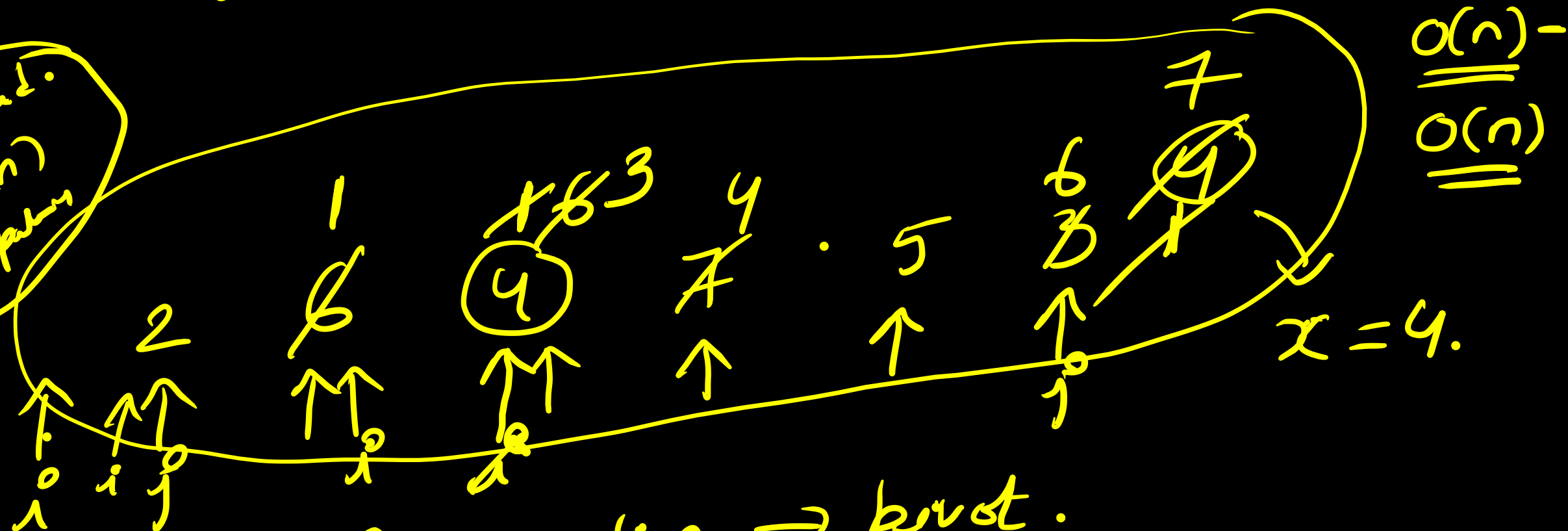
$$= \underline{\underline{O(n \log n)}} \rightarrow \underline{\underline{\text{master, then.}}}$$

The median of n elements can be found in $O(n)$ time.

What is the TC of QS, in which median is selected as pivot.

$O(n)$ - med.
 $T(n) + T(n)$
 $O(n)$ - pivot

$O(n) + O(n)$



$O(n) \rightarrow \text{median} \rightarrow \text{pivot.}$

$$T(n) = \begin{cases} T(n/2) + T(n/2) \\ + O(n) + O(n) \end{cases}$$

$$= \begin{cases} 2T(n/2) + O(n) \end{cases}$$

$$= \underline{\underline{n \log n}}$$



GATE In quick sort, for sorting n elements, the $(n/4)^{\text{th}}$ smallest element is selected as pivot using $O(n)$ time algorithm. What is the worst case time complexity of QS?

$O(n) \rightarrow \underline{\underline{n/4^{\text{th}} \text{ element}}}$

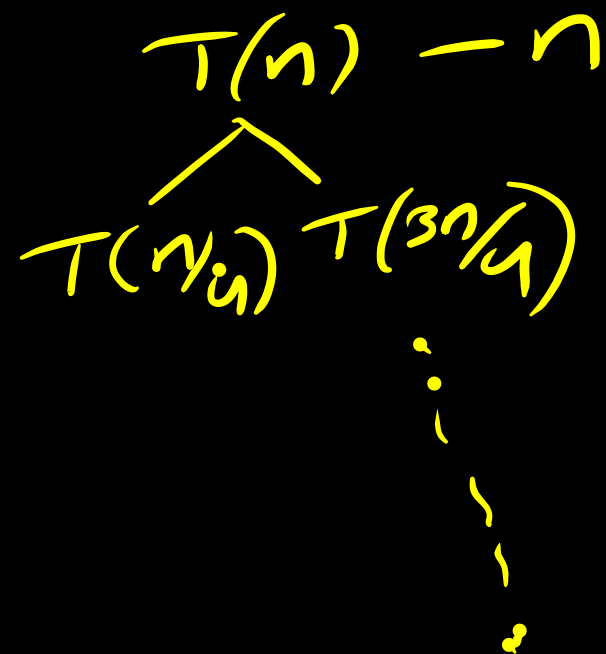
$O(n) \rightarrow \text{partition}$

$$T(n) = T(n/4) + T(\cancel{3n/4} \ 3n/4) + O(n)$$

~~1:3~~ 1:3

1:9
1:99
1:999

} $O(n \log n)$



Ques: ~~Ques~~ In QS if elements are $1, 2, 3, \dots, n \rightarrow T_1$
 $n, n-1, n-2, \dots, 1 \rightarrow T_2$

T_1, T_2

$=$

→ In QS by using an $O(n)$ partition algo, we always partition array into one part $1/5 n$ other part $4n/5$, then what is the TC?

$$\begin{aligned} T(n) &= \cancel{\frac{1}{5} T(n)} + T(4n/5) + O(n). \\ &= O(n^{\log_{5/4} n}) = \underline{\underline{O(n \log n)}} \end{aligned}$$